Reading Notes: Damage Propagation Modeling for Aircraft Engine Run-to-Failure Simulation

Saxena, Goebel, Simon, Eklund — PHM'08 — *In-depth study edition*

Contents

1 The Problem Being Solved **2**

2 PHM: Where Prognostics Sits **2**

3 The Inverse Problem: Why PHM Is Hard **3**
3.1 The forward problem 4
3.2 The inverse problem 4

4 The Tool: C-MAPSS **4**
4.1 What C-MAPSS is 4
4.2 Inputs: health parameter modifiers 5
4.3 Outputs: sensor measurements and margins 5
4.4 Closed-loop operation and the control problem 6

5 How Degradation Was Represented **6**
5.1 The two health variables 6
5.2 Why this is a reduced-order model 6

6 Damage Propagation Models **6**
6.1 Arrhenius model 6
6.2 Coffin-Manson model 7
6.3 Eyring model 7
6.4 The common thread: exponential behavior 7
6.5 The chosen model: generalized exponential wear 7

7 The Health Index: Defining Failure **9**
7.1 From module health to overall engine health 9
7.2 Failure as geometry 10
7.3 Dependence on operating conditions 10

8 The Application Scenario and Noise Architecture **10**
8.1 One measurement per flight 10
8.2 The layered noise architecture 11

9 The Simulation Pipeline **12**

10 Competition Dataset Structure **13**
10.1 What was generated 13
10.2 What participants received and did not receive 13

11 Performance Evaluation and Scoring **14**
11.1 The asymmetric scoring function 14
11.2 Why asymmetric? 14
11.3 The aggregate score and its limitation 15

12 The Bigger Picture: What This Paper Is **15**

13 The Dataset: Structure and Files **15**
13.1 Four sub-datasets 15
13.2 File structure: 12 files total 16
13.3 The 26-column format 17
13.4 Training RUL: what must be computed 18
13.5 The multi-condition problem 18
13.6 What the data does not contain 18

Appendix: Equation Reference **19**

The Problem Being Solved

The paper opens with a simple and uncomfortable fact: data-driven prognostics — the field of predicting when a machine will fail — cannot progress without *run-to-failure data*. And run-to-failure data for aircraft engines is nearly impossible to obtain.

Real engines do not fail on a convenient schedule. Operators who accumulate failure records keep them private. Fielded systems are often under-instrumented. The result is that researchers have no shared ground truth against which to test and compare their algorithms.

> **The core question the paper answers:** Can we use a physics-based simulator to manufacture believable run-to-failure data — good enough to train and benchmark prognostics algorithms?

The answer the authors demonstrate is yes. But the answer requires solving three sub-problems in sequence:

1. Finding a simulator flexible enough to accept injected degradation
2. Deciding how to model the evolution of that degradation over time
3. Defining when the simulated engine has "failed"

Each of the paper's sections addresses one of these in turn.

PHM: Where Prognostics Sits

Before going further, it helps to understand what **PHM** — Prognostics and Health Management — actually is, and where prognostics fits inside it.

PHM is a five-layer ecosystem, shown in Figure 1. Each layer answers a more demanding question than the one below it.

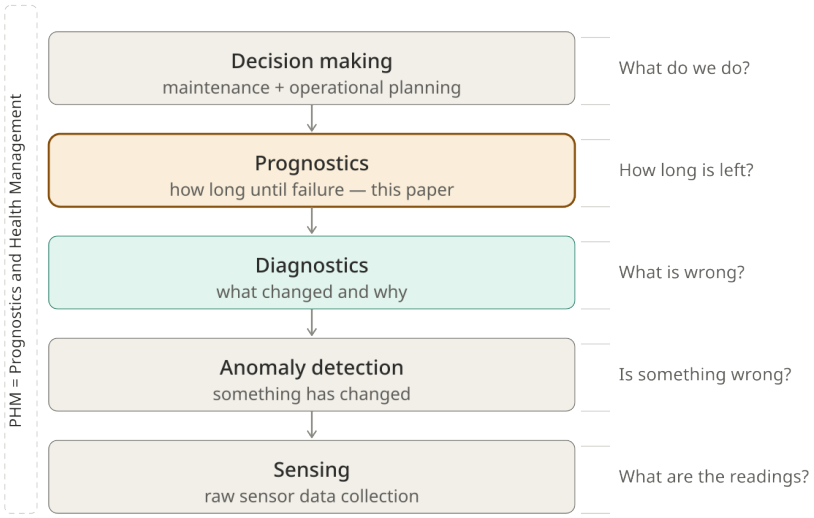

Figure 1: The PHM ecosystem. Prognostics (highlighted) sits at layer 4, between diagnostics and operational decision-making. This paper scopes exclusively to that layer.

The paper explicitly defines prognostics as *estimation of remaining useful life (RUL)* — not fault detection, not fault identification. This scoping decision is important: everything in the paper is designed to support RUL estimation, and nothing else.

Why does the scoping matter? A diagnostic algorithm asks "what is wrong?" A prognostic algorithm asks "how long is left?" These are different inference problems requiring different data, different models, and different evaluation metrics. The C-MAPSS dataset was built for the second question only. Using it for the first requires caution.

The larger PHM chain beyond prognostics — maintenance decisions and operational planning — is implied by the scoring function in Section VII of the paper. The asymmetric penalty (predicting late is worse than predicting early) only makes sense if we remember that the *purpose* of RUL prediction is to inform decisions with real safety consequences.

The Inverse Problem: Why PHM Is Hard

The most important conceptual framing in the paper is one it never explicitly names: PHM is an **inverse problem**.

Figure 2 shows the two directions of the problem.

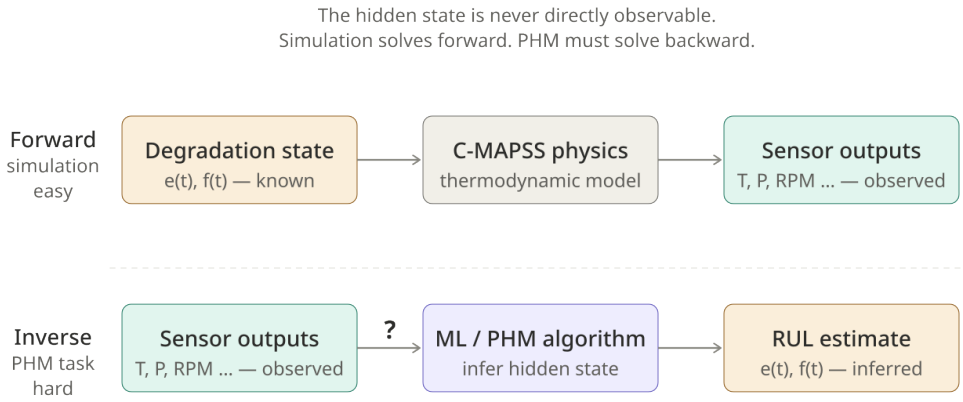

Figure 2: The forward problem (simulation) is easy. The inverse problem (PHM inference) is hard. The hidden degradation state $e(t), f(t)$ is never directly observable — only its projection through engine physics onto sensor signals can be seen.

The forward problem

In simulation, the direction is:

$$(e(t),\ f(t)) \xrightarrow{\text{C-MAPSS physics}} \text{sensor outputs}$$

This is straightforward. You set the health parameters, run the model, and read off the sensor values. The mapping is deterministic and well-defined.

The inverse problem

In real PHM deployment, the direction is reversed:

$$\text{sensor outputs} \xrightarrow{?} (e(t),\ f(t)) \xrightarrow{?} \text{RUL}$$

You observe only the sensor signals. You must infer the hidden degradation state, and from that infer how much life remains. This is harder for several reasons:

- The mapping from (e, f) to sensors is *nonlinear*
- The controller actively compensates for degradation, obscuring it in the sensor record
- The true (e, f) values are never revealed to the algorithm
- Noise at multiple stages further blurs the signal

The deep implication: the C-MAPSS dataset is designed so that the algorithm *cannot* solve the inverse problem directly. It must learn the statistical relationship between sensor trajectories and RUL from the training data. This is precisely what makes it a meaningful benchmark.

The Tool: C-MAPSS

What C-MAPSS is

C-MAPSS (Commercial Modular Aero-Propulsion System Simulation) is a NASA MATLAB/Simulink model of a large commercial turbofan engine (90,000 lb thrust class). It was the right choice for

this work because it meets the key requirement: it accepts *health parameter inputs* that allow the user to inject degradation, and it produces *sensor outputs* that respond accordingly.

Figure 3 shows the engine's five rotating modules and the overall flow path.

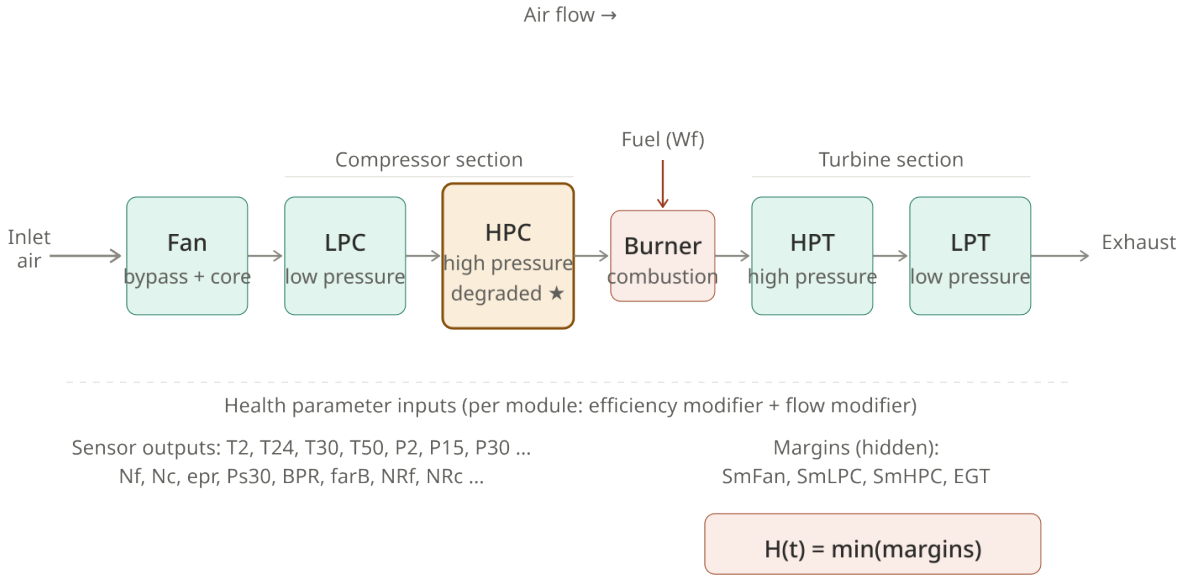

Figure 3: C-MAPSS turbofan engine schematic. Air enters at the fan and exits through the nozzle, passing through LPC, HPC, the combustor, HPT, and LPT in sequence. The HPC module (amber) is the one degraded in the competition dataset. Health modifiers enter each module; 21 sensor outputs are recorded; hidden margins feed the health index.

Inputs: health parameter modifiers

C-MAPSS accepts 14 inputs. Thirteen of them are health modifiers — efficiency, flow, and pressure-ratio modifiers for each of the five rotating components. The fourteenth is fuel flow.

The health modifiers are the "knobs" of the degradation injection. By reducing the HPC efficiency modifier and the HPC flow modifier, the authors simulate a compressor that is progressively losing performance.

> **Important constraint:** for the competition dataset, only HPC degradation was simulated. Fan, LPC, HPT, and LPT were left at healthy values. This was an intentional simplification to keep the benchmark focused.

Outputs: sensor measurements and margins

C-MAPSS produces 58 outputs. The authors selected 21 for the dataset, covering temperatures ($T_2, T_{24}, T_{30}, T_{50}$), pressures ($P_2, P_{15}, P_{30}, P_{s30}$), speeds ($N_f, N_c, NR_f, NR_c$), flow ratios ($\phi, BPR, farB$), bleed enthalpies, and demanded values.

Four additional outputs — T_{48} (EGT), $SmFan$, $SmLPC$, $SmHPC$ — are used internally to compute the health index but are *not* provided to competition participants. This is deliberate: participants must infer health from the 21 observable sensors, not from the margins directly.

Closed-loop operation and the control problem

A critical detail: the authors ran C-MAPSS in **closed-loop** mode, meaning the engine's built-in controller was active during all simulations. This has a significant consequence.

The controller's job is to maintain commanded fan speed and to prevent the engine from exceeding design limits. As the HPC degrades, the controller compensates — it adjusts fuel flow to maintain thrust. This means the sensor signals do not directly reveal the degradation. Instead, they show a mixture of the degradation effect and the controller's response.

In Figure 3, note the feedback loop implied by the closed-loop architecture. The EGT rises as HPC efficiency drops not just because the thermodynamics change, but because the controller demands more fuel to compensate. Health monitoring in a closed-loop system is inherently harder than in an open-loop one.

How Degradation Was Represented

The two health variables

The paper represents the health of any engine module using just two numbers:

Variable	Physical meaning
$e(t)$	Efficiency — how well the module converts input work into useful output. Degradation reduces this.
$f(t)$	Flow capacity — how much air mass passes through the module per unit time. Degradation generally reduces this too.

These two numbers define the *hidden state* of the engine. Every sensor output is a function of $(e(t), f(t))$ — and the flight conditions.

Why this is a reduced-order model

Real compressor degradation involves thousands of physical variables: blade surface roughness, tip clearance, seal wear, deposit buildup, and more. The authors deliberately discard all of this microscopic detail and retain only two aggregate parameters per module.

This is a **reduced-order health model**: a deliberate compression of the full physical state into the minimum number of variables needed to produce realistic sensor responses. It is not physically perfect. It does not need to be. Its purpose is to create a *controllable degradation-to-sensor mapping* — and for that purpose, two variables per module are sufficient.

Damage Propagation Models

Having chosen C-MAPSS as the simulator, the next question is: how should $(e(t), f(t))$ evolve over time? The paper reviews three classical degradation models before arriving at its own.

Arrhenius model

Originally developed for chemical and diffusion failure processes, the Arrhenius model gives the time to failure as:

$$t_f = A\, e^{\,\Delta H/kT} \tag{1}$$

where T is temperature, k is Boltzmann's constant, A is a scaling factor, and ΔH is the activation energy. The physical intuition is simple: higher temperature means more atoms have enough energy to cross the activation barrier, so degradation processes run faster.

The key insight from Arrhenius: *degradation rate is exponentially sensitive to temperature.* This is why turbine components — which run at extreme temperatures — degrade faster than compressor components.

Coffin-Manson model

More relevant to mechanical fatigue, the Coffin-Manson model gives the cycles to failure as:

$$N_f = A\, f^{-\alpha}\, \Delta T^{-\beta}\, G(T_{\max}) \tag{2}$$

where f is cycling frequency, ΔT is the temperature range per cycle, $G(T_{\max})$ is an Arrhenius term at peak temperature, and α, β are exponents. This model captures the damage accumulated through repeated heating and cooling cycles — the dominant failure mode for rotating machinery.

The key insight: *each thermal cycle deposits damage.* The damage accumulates until the component fails. Number of cycles to failure depends on both the severity and frequency of the cycling.

Eyring model

The most general of the three, the Eyring model handles multiple simultaneous stresses:

$$t_f = AT^{\alpha} \exp\left(\frac{\Delta H}{kT} + \left(B + \frac{C}{T}\right) S_1 + \left(D + \frac{E}{T}\right) S_2\right) \tag{3}$$

where S_1, S_2 are additional stresses (voltage, pressure, vibration, etc.) and the constants determine how stresses interact with temperature. It is the right model when multiple degradation mechanisms act simultaneously, but it requires many parameters to be calibrated.

The common thread: exponential behavior

All three models share a structural feature: the rate of degradation *accelerates* over time. Small initial wear creates additional stress, which causes faster wear, which creates more stress. This positive feedback loop produces the characteristic exponential-like growth in damage rate:

small wear → more stress
more stress → faster wear
faster wear → even more stress
→ runaway degradation

The key abstraction: the authors observe that all three models produce exponential macro-level behavior, regardless of the microscopic mechanism. So rather than choosing one physical model and fitting its parameters, they simply adopt the exponential form directly. The specific physics is discarded; the correct macroscopic *shape* of degradation is retained.

The chosen model: generalized exponential wear

Starting from a generalized wear equation $w = Ae^{B(t)}$ with an upper wear threshold th_w, and recasting in terms of health (normalized remaining safe margin), the paper arrives at:

$$h(t) = 1 - \exp\Big\{ at^b \Big\} \tag{4}$$

To allow the simulation to start at an arbitrary point in the wear space (modeling manufacturing variation and prior usage), an initial deterioration term d is added:

$$h(t) = 1 - d - \exp\Big\{ at^b \Big\} \tag{5}$$

Since both efficiency and flow degrade (with potentially different rates and shapes), each gets its own version of this equation:

$$e(t) = 1 - d_e - \exp\Big\{ a_e\, t^{b_e} \Big\} \tag{6}$$

$$f(t) = 1 - d_f - \exp\Big\{ a_f\, t^{b_f} \Big\} \tag{7}$$

The parameters of these equations are summarized in Table 1.

Table 1: Parameters of the health equations and their roles.

Parameter	Range	Role
d	$\leq 1\%$	Initial deterioration. Drawn randomly per engine, models manufacturing variation and prior wear.
a_k	$[0.001, 0.003]$	Degradation rate. Higher a means faster deterioration.
b_k	$[1.4, 1.6]$	Shape exponent. Controls how quickly the exponential accelerates. $b > 1$ produces superlinear growth.
t	≥ 0	Engine cycles. One cycle $\approx$ one flight.

Figure 4 shows what this produces: a family of curves that all start near $h = 1$ and drift toward $h = 0$, but at different rates and with different shapes depending on their randomly chosen parameters.

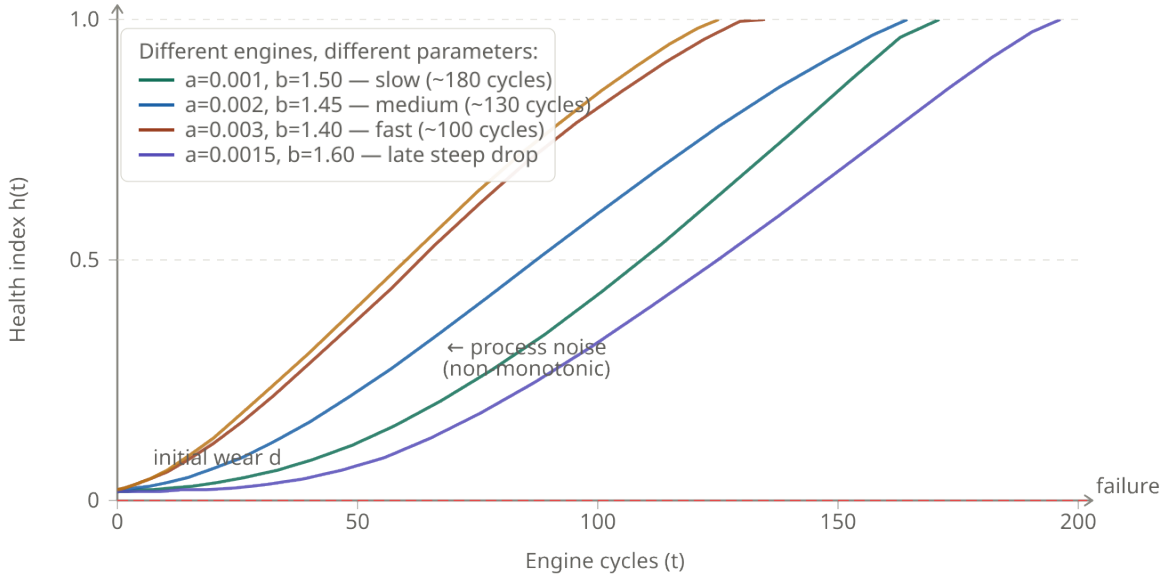

Figure 4: A family of degradation trajectories $h(t)$ for five engines with different parameter sets. All share the same functional form but reach failure at different cycle counts. The slow engine lives ∼180 cycles; the fast one ∼100. In the real dataset, small process noise makes each trajectory slightly non-monotonic.

The Health Index: Defining Failure

From module health to overall engine health

The efficiency and flow trajectories $e(t)$ and $f(t)$ describe the hidden degradation state. But when does the engine actually fail? The paper defines failure operationally: an engine fails when it can no longer operate safely within its design limits under any possible flight condition.

This is captured through **operability margins** — how far the engine is from violating a safety constraint. Four margins are tracked:

Margin	What it measures
m_{Fan}	Distance from fan stall. Stall = sudden loss of compression, catastrophic.
m_{LPC}	Distance from LPC stall. Same risk.
m_{HPC}	Distance from HPC stall. Most relevant given the HPC is the degraded module.
m_{EGT}	Distance from maximum exhaust gas temperature. Overtemperature causes turbine blade damage.

Each margin is normalized to $[0, 1]$. The overall health index is the minimum:

$$H(t) = \min\big(m_{\text{Fan}},\ m_{\text{LPC}},\ m_{\text{HPC}},\ m_{\text{EGT}}\big) \tag{8}$$

Why the minimum? The engine fails as soon as *any one* safety boundary is violated. A chain breaks at its weakest link. $H(t) = 0$ when the first margin is exhausted — regardless of how healthy the engine is on all other margins.

Failure as geometry

Figure 5 provides the geometric interpretation. The hidden degradation state is a point in the two-dimensional (e loss, f loss) space. As the engine ages, this point moves away from the healthy origin (bottom-left) toward the failure region (upper-right).

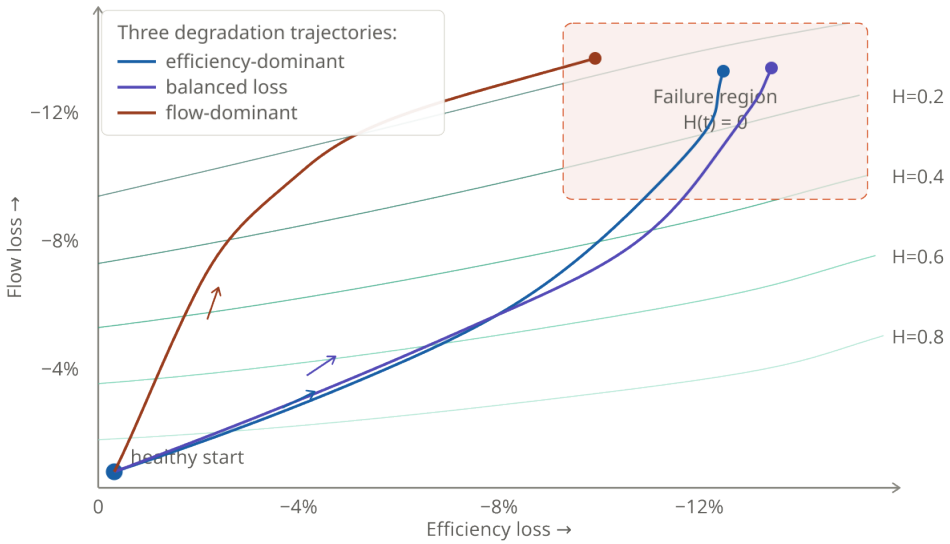

Figure 5: Degradation as movement through latent health space. Contour lines show equal-health-index surfaces. Three trajectories with different directions reach the failure boundary at different points. The trajectory that moves primarily along the efficiency axis reaches failure differently from one that moves primarily along the flow axis.

The contour lines in Figure 5 represent the health landscape. Each contour is a locus of (e, f) pairs that produce the same health index. The failure boundary is the contour at $H = 0$.

The direction of the degradation trajectory matters. An engine that loses efficiency faster than flow will cross a different boundary (likely the EGT margin) than one that loses flow faster (likely the HPC stall margin). This is why the paper discusses Figures 7 and 8 separately in the original: two response surfaces, one trajectory family.

Dependence on operating conditions

An important subtlety: the margins are not constant. They depend on flight conditions — altitude, Mach number, and throttle resolver angle (TRA). An engine operating at hot-day takeoff faces tighter margins than the same engine cruising at altitude.

The paper simulated six different flight condition combinations. The health index computation considers the worst-case condition: if the engine would violate a margin under *any* simulated condition, its health index is zero. This ensures the dataset reflects realistic operational risk.

The Application Scenario and Noise Architecture

One measurement per flight

The simulation models each engine flight as a single snapshot taken at cruise conditions. This reflects real operational practice: health monitoring systems typically record one representative

measurement per flight, rather than continuous high-rate data.

The effects of between-flight maintenance are *not* explicitly modeled but are incorporated as process noise — small random variations that can occasionally improve performance, making the efficiency/flow trajectory locally non-monotonic. This is physically realistic: a cleaning event or minor adjustment can temporarily recover a few percent of performance.

The layered noise architecture

This is one of the most carefully designed aspects of the dataset. Four distinct noise sources are stacked, each added at a different stage of the simulation pipeline. Figure 6 shows the full stack.

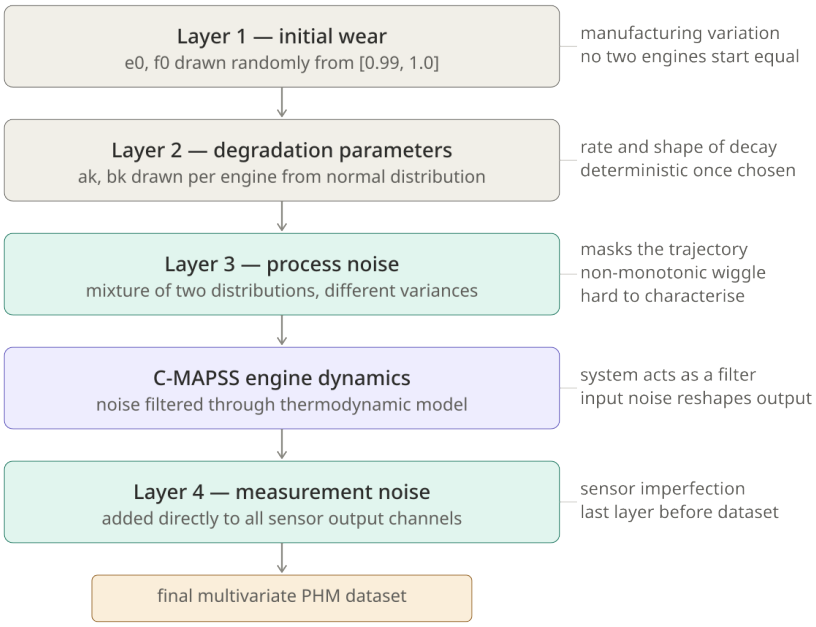

Figure 6: The four-layer noise architecture of the C-MAPSS dataset. Each layer introduces a different kind of uncertainty. The layers are not independent — process noise enters the simulation and is reshaped by the engine dynamics before reaching the sensor outputs.

The four layers are:

Layer 1 — Initial wear. The starting values e_0 and f_0 are drawn uniformly from $[0.99, 1.0]$. Every simulated engine begins slightly below perfect health, just as real engines leave the factory with small imperfections.

Layer 2 — Degradation parameters. For each engine, the trajectory parameters a_k and b_k are drawn from a normal distribution. This produces a *deterministic* trajectory for each engine — but each engine has a different one. Given e_0, f_0, a_k, b_k, the health trajectory is fully determined.

Layer 3 — Process noise. The deterministic trajectory is then *masked* by a mixture of two normal distributions with slightly different variances. This is the subtlest noise layer. A mixture model is harder to characterize than a single Gaussian. The choice is deliberate: it forces algorithms to deal with non-trivial uncertainty structure, rather than simple additive Gaussian noise. This layer also models the between-flight maintenance effects.

Layer 4 — Measurement noise. After the noisy health trajectory is fed through C-MAPSS and steady-state sensor outputs are computed, a final independent random noise term is added

to each sensor channel. This models sensor imperfection: quantization error, calibration drift, electrical noise.

Why all four layers? Each layer creates a different challenge for the PHM algorithm. Layer 1 means no two engines start at the same health state. Layer 2 means no two engines age at the same rate. Layer 3 means the trajectory cannot be recovered by simple filtering. Layer 4 means even a perfect model of the engine dynamics would not give clean health estimates from sensor data.

An algorithm that handles one layer but not the others will fail in real deployment. The dataset tests all four simultaneously.

The Simulation Pipeline

The complete data generation process — from parameter initialization to a finished run-to-failure trajectory — is shown in Figure 7.

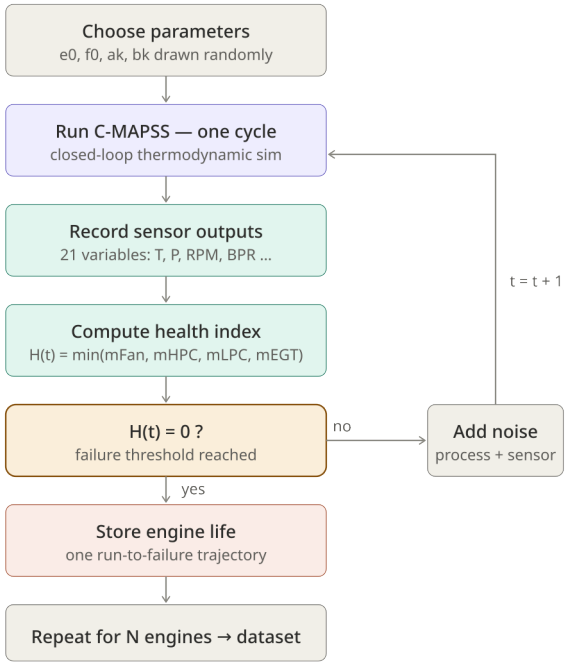

Figure 7: The C-MAPSS simulation loop. Each pass through the main loop generates one cycle of sensor data for one engine. The loop continues until the health index reaches zero. The entire loop is repeated for each engine in the dataset.

The data generation procedure in precise terms:

1. Draw $e_0, f_0 \in [0.99, 1.0]$ (initial wear).
2. Draw $a_k \in [0.001, 0.003]$ and $b_k \in [1.4, 1.6]$ for both efficiency and flow (degradation parameters).
3. Compute the health trajectory $e(t), f(t)$ from Equations (6)–(7).
4. Mask the trajectory with mixture-distribution process noise.

5. Feed the noisy $(e(t), f(t))$ to C-MAPSS under a randomly chosen cruise condition at each step.
6. Record the 21 sensor outputs once steady state is reached.
7. Add independent measurement noise to all 21 channels.
8. Compute the health index $H(t)$ from the four margins.
9. If $H(t) = 0$: stop. This is the failure point.
10. Otherwise: increment t and repeat from step 4.

One complete run of this procedure produces one *run-to-failure trajectory* — a time series of 21 sensor readings from some initial cycle count to the failure cycle.

The procedure is repeated hundreds of times to build the full dataset, with independent parameter draws each time.

Competition Dataset Structure

What was generated

The final dataset consists of trajectories simulating HPC degradation under six different combinations of altitude, TRA, and Mach number. The six conditions were chosen to span a realistic range of operational scenarios without being exhaustively comprehensive.

The dataset is divided into three sets with importantly different properties:

Set	Properties
Training	Full trajectories from initial condition to failure. The algorithm sees the complete engine life and knows when failure occurred.
Test	Trajectories *truncated* at some point before failure. RUL ranges from 10 to 150 cycles. Score feedback given after each submission.
Validation	Trajectories truncated before failure. RUL ranges from 6 to 190 cycles — a wider distribution than the test set. No feedback during competition.

The wider RUL range in the validation set was a hidden robustness test. Algorithms that overfit to the test set distribution (RUL ≤ 150) would fail on validation cases with RUL up to 190. This was not announced to participants, making the validation set an honest out-of-distribution test.

What participants received and did not receive

Participants received the 21 sensor readings per cycle for each engine. They did *not* receive:

- The health index $H(t)$
- The four margin values (m_{Fan}, m_{LPC}, m_{HPC}, m_{EGT})
- The degradation parameters (a_k, b_k, d_e, d_f)
- The hidden state $(e(t), f(t))$

All of this had to be inferred from the 21 observable sensor channels.

Performance Evaluation and Scoring

The asymmetric scoring function

For any RUL prediction, define the prediction error as:

$$d = \hat{t}_{\text{RUL}} - t_{\text{RUL}} \quad \text{(estimated minus true)}$$

The aggregate score across n test units is:

$$s = \begin{cases} \displaystyle\sum_{i=1}^{n} e^{-d_i/10} - 1 & \text{if } d_i < 0 \quad \text{(early prediction)} \\ \displaystyle\sum_{i=1}^{n} e^{d_i/13} - 1 & \text{if } d_i \geq 0 \quad \text{(late prediction)} \end{cases} \tag{9}$$

Figure 8 shows this function graphically.

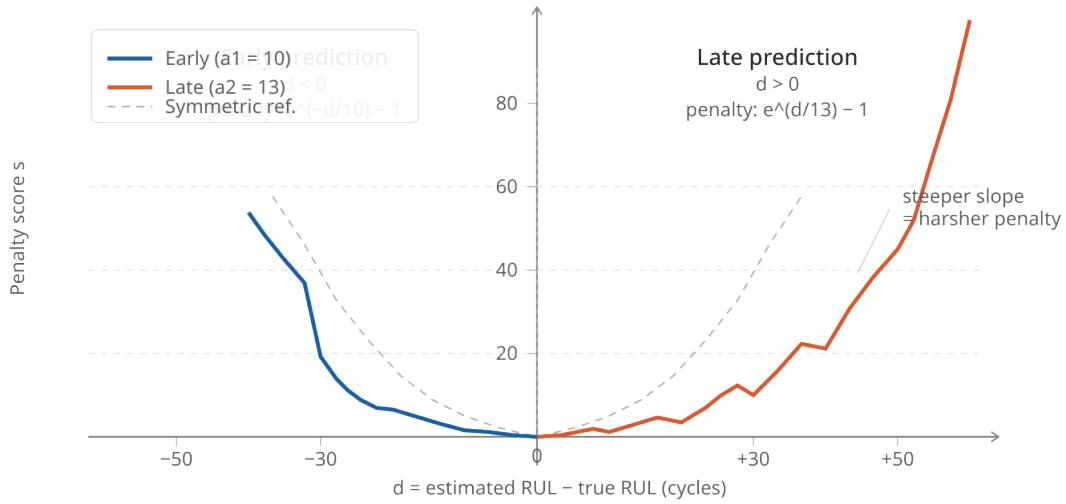

Figure 8: The asymmetric scoring function. Both sides grow exponentially with error magnitude. The late-prediction side (red, denominator 13) is steeper than the early-prediction side (blue, denominator 10), encoding the aviation safety preference: it is better to predict early and schedule unnecessary maintenance than to predict late and risk an in-flight failure. The dotted line shows what a symmetric function would look like.

Why asymmetric?

The asymmetry encodes a value judgment:

Error type	Consequence	Penalty
Late ($d > 0$)	Engine may fail before maintenance	Severe
Early ($d < 0$)	Unnecessary maintenance cost	Moderate

In aviation, the first consequence is unacceptable. The second is costly but manageable. The denominators $a_1 = 10$ and $a_2 = 13$ translate this asymmetry into a quantitative penalty structure.

The scoring function is a safety statement encoded as math. Choosing a_1 and a_2 is equivalent to specifying how many dollars of unnecessary maintenance is worth risking to avoid one unscheduled engine removal. PHM practitioners should treat these parameters as domain-specific design choices, not universal constants.

The aggregate score and its limitation

The score in Equation (9) aggregates across all test units using a sum. This creates an important failure mode: two algorithms with completely different prediction behaviors can produce identical aggregate scores.

The paper illustrates six prediction scenarios where the aggregate score is always 6, but the correlation between predicted and true RUL ranges from 0.0 to 1.0. An algorithm with correlation 0.0 — whose predictions have no relationship to ground truth whatsoever — can score identically to an algorithm with perfect correlation, depending on how the errors happen to cancel.

The authors' conclusion: aggregate score should be supplemented by a correlation metric to ensure *consistent* prediction behavior across the fleet, not just a lucky average.

The Bigger Picture: What This Paper Is

The paper is best understood as three things simultaneously:

A methodology paper. It documents the architecture of a physics-based degradation simulation pipeline. Every design choice — the exponential wear model, the minimum-margin health index, the layered noise — is explained and justified. Other researchers can apply the same architecture to different systems.

A dataset paper. It is the primary citation for the C-MAPSS dataset, which became one of the most widely used benchmarks in predictive maintenance research. Understanding the paper is understanding the dataset's assumptions and limitations.

An early digital twin. The combination of physics-based simulation + parameterized degradation injection + sensor output generation is structurally identical to what is now called a digital twin. The paper was published in 2008. The vocabulary caught up later.

In one sentence: The paper answers the question "how do we generate believable run-to-failure data for aircraft engines?" by building a controllable physics-to-sensor mapping, injecting exponential degradation into the hidden state, and observing the resulting sensor trajectories through multiple layers of realistic noise.

The Dataset: Structure and Files

Four sub-datasets

The C-MAPSS release is not a single dataset but a family of four, each generated under different simulation conditions. They share the same 26-column format and the same engine model, but differ in two independent axes of difficulty: the number of operating conditions and the number of active fault modes.

Figure 9 shows the complete matrix.

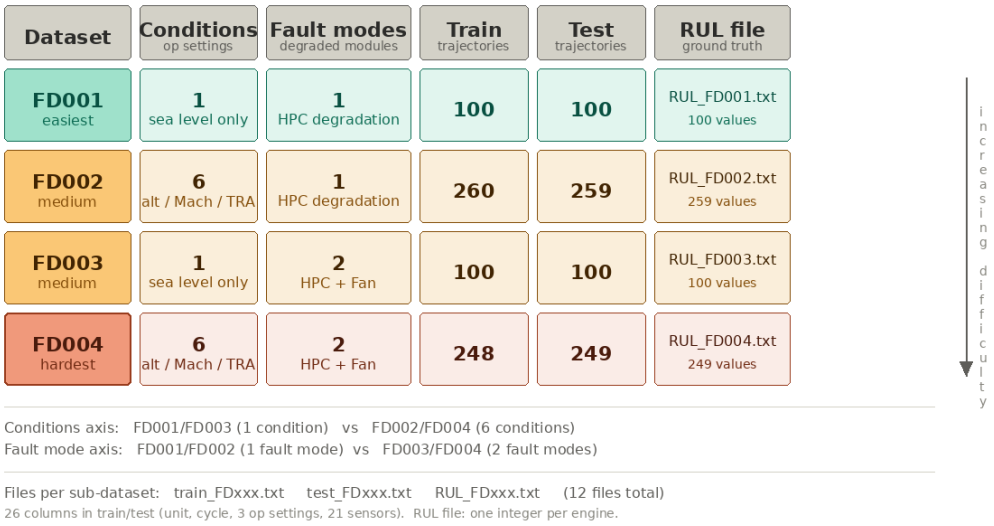

Figure 9: The four C-MAPSS sub-datasets as a difficulty matrix. Conditions and fault modes are independent axes. FD002 and FD003 are both of medium difficulty but are hard in different ways. FD004 combines both challenges simultaneously.

The two axes of difficulty are worth keeping separate in your mind:

Conditions axis (FD001/FD003 vs FD002/FD004). With one condition (sea level), raw sensor values are directly comparable across cycles and engines. With six conditions (altitude, Mach, TRA combinations), the same sensor reading means different things at different operating points. A fan temperature at cruise altitude is not the same as one at sea level takeoff. Any algorithm applied to FD002 or FD004 must normalize by condition before the sensor signals carry meaningful health information.

Fault mode axis (FD001/FD002 vs FD003/FD004). With one fault mode (HPC only), the degradation source is fixed. With two fault modes (HPC and Fan), either module may be degrading — or both simultaneously. The algorithm receives no label indicating which fault mode is active. It must infer health trajectory from sensor signals alone, without knowing the identity of the degrading component.

> **Practical consequence:** most published research reports results on FD001. It is the easiest, the most reproducible, and the most comparable across papers. A result on FD001 alone should be interpreted cautiously — it says nothing about robustness to operating condition variability (FD002) or fault mode ambiguity (FD003/FD004).

File structure: 12 files total

Each sub-dataset produces exactly three files:

File	Contents
`train_FDxxx.txt`	Full run-to-failure trajectories. Each engine runs from initial wear to failure. The last row per engine is the failure cycle. **RUL must be computed** from the data: $\text{RUL}(t) = t_{\max} - t$.
`test_FDxxx.txt`	Truncated trajectories. Each engine's time series ends at some point before failure. The last row is not the failure cycle. **RUL is unknown** and must be predicted.
`RUL_FDxxx.txt`	Ground truth. One integer per engine, giving the true RUL at the last observed cycle of that engine in the test set. Used only for evaluation.

Four sub-datasets × three files = **12 files total**.

The 26-column format

All `train` and `test` files share the same structure: 26 space-separated columns, no header row, one cycle per row. Table 2 gives the complete column reference.

Table 2: Complete column reference for C-MAPSS train and test files.

Col	Name	Units	Description
1	`unit_number`	—	Engine identifier. Resets per file.
2	`time_cycles`	cycles	Cycle counter per engine. Starts at 1.
3	`op_setting_1`	ft	Altitude.
4	`op_setting_2`	—	Mach number.
5	`op_setting_3`	—	Throttle resolver angle (TRA).
6	`T2`	°R	Total temperature, fan inlet.
7	`T24`	°R	Total temperature, LPC outlet.
8	`T30`	°R	Total temperature, HPC outlet.
9	`T50`	°R	Total temperature, LPT outlet.
10	`P2`	psia	Pressure, fan inlet.
11	`P15`	psia	Total pressure, bypass duct.
12	`P30`	psia	Total pressure, HPC outlet.
13	`Nf`	rpm	Physical fan speed.
14	`Nc`	rpm	Physical core speed.
15	`epr`	—	Engine pressure ratio (P_{50}/P_2).
16	`Ps30`	psia	Static pressure, HPC outlet.
17	`phi`	pps/psi	Fuel flow / P_{s30} ratio.
18	`NRf`	rpm	Corrected fan speed.
19	`NRc`	rpm	Corrected core speed.
20	`BPR`	—	Bypass ratio.
21	`farB`	—	Burner fuel-air ratio.
22	`htBleed`	—	Bleed enthalpy.
23	`Nf_dmd`	rpm	Demanded fan speed.
24	`PCNfR_dmd`	rpm	Demanded corrected fan speed.
25	`W31`	lbm/s	HPT coolant bleed.
26	`W32`	lbm/s	LPT coolant bleed.

The grouping is meaningful: columns 1–2 are identifiers, columns 3–5 are operating conditions, and columns 6–26 are the 21 sensor measurements from the paper's Table 2.

The `RUL_FDxxx.txt` files contain a single column of integers with no header, one value per engine in the test set.

Training RUL: what must be computed

The training files do not contain a RUL column. To create training targets, the standard procedure is:

1. Group rows by `unit_number`.
2. For each engine, find $t_{\max}$ — the last `time_cycles` value (failure cycle).
3. For each row: $\text{RUL} = t_{\max} - t_{\text{current}}$.

This gives RUL = 0 at the failure cycle, counting upward backward. In practice many researchers apply a **piece-wise linear RUL cap**: RUL is clamped to some maximum value (commonly 125 or 130 cycles) for early cycles, creating a flat-then-decreasing target shape. The rationale is that predicting very large RUL values precisely is uninformative and destabilises model training. This is a preprocessing decision not specified in the dataset or paper.

The multi-condition problem

For FD002 and FD004, the six operating conditions produce dramatically different absolute sensor values. Fan temperature T_2 at sea level differs by hundreds of degrees Rankine from T_2 at 40,000 ft. Feeding raw values to an ML model on these sub-datasets trains the model to recognise operating points, not degradation.

The standard fix is **condition-wise normalisation**:

1. Cluster cycles by operating condition using the three `op_setting` columns. Six clusters emerge naturally for FD002/FD004.
2. Within each cluster, compute the mean and standard deviation of each sensor across the healthy (early) cycles of all engines.
3. Normalise each sensor reading relative to its cluster-specific statistics.

After this step, sensor values express deviation from healthy baseline at that condition, not absolute thermodynamic state. Only then do the signals carry interpretable health information across different operating points.

For FD001 and FD003 (single condition), this step is simpler — a single global normalisation suffices — but the principle is the same.

What the data does not contain

It is worth writing down explicitly what is absent, because the paper describes quantities that never appear in the files:

Absent from files	Must be derived or ignored
Health index $H(t)$	Not provided; infer from sensors
Margin values (SmFan, SmHPC, ...)	Hidden; used only in simulation
Hidden state $(e(t), f(t))$	Never revealed
Fault mode label	Not provided (FD003/FD004)
Operating condition label	Must be inferred from op_settings
RUL column in training files	Must be computed from max cycle

The algorithm sees only the 26 columns. Everything else is the inference problem.

Appendix: Equation Reference

Equation	Formula	Where used
Arrhenius	$t_f = Ae^{\Delta H/kT}$	Background (Sec. 6.1)
Coffin-Manson	$N_f = Af^{-\alpha}\Delta T^{-\beta}G(T_{\max})$	Background (Sec. 6.2)
Eyring	$t_f = AT^{\alpha}e^{(\cdots)}$	Background (Sec. 6.3)
Health (basic)	$h(t) = 1 - \exp\{at^b\}$	Core model
Health (with d)	$h(t) = 1 - d - \exp\{at^b\}$	Actual simulation
Health index	$H(t) = \min(m_{\text{Fan}}, m_{\text{HPC}}, m_{\text{LPC}}, m_{\text{EGT}})$	Failure criterion
Score (early)	$s = \sum e^{-d/10} - 1$	Competition metric
Score (late)	$s = \sum e^{d/13} - 1$	Competition metric